МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №1
по курсу “Компьютерная графика”

Основы 3D графики

Выполнил: Н. С. Фомичев
Группа: М8О-309Б-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель – познакомиться с основами 3D-графики, построением простых 3D-объектов, проекцией на 2D-плоскость, а также научиться работать с матрицами перспективы и ортографической проекцией.

Вариант задания №1 – куб.

Метод решения

Положение каждой вершины куба проходит через стандартную цепочку преобразований:

clip = P * V * M * v_local

где v_local - координаты вершины в локальной системе куба

Model-матрица M собирается как T * R_y * R_x * R_z * S. T - трансляция (currentPosition() = ручная позиция + смещение по траектории), R_y, R_x, R_z - повороты по Эйлеру через glm::rotate, S - неравномерное масштабирование по трём осям.

View-матрица V - обратная к матрице камеры.

Projection-матрица P - одна из двух. Перспективная: glm::perspective(fov, aspect, near, far) - классическая однородная проекция tan(fov/2); ортографическая: glm::ortho(-h*aspect, h*aspect, -h, h, near, far). После применяется P[1][1] *= -1 - Vulkan-овский clip-space имеет ось Y направленную вниз, в отличие от OpenGL, под который задумывался GLM.

Траектория - трёхмерная кривая Лиссажу: pos_i(t) = R * sin(2π * (f_i * t + φ_i)) по каждой оси независимо. Разные частоты порождают незамкнутые/замкнутые траектории разной сложности. Время t накапливается только при Play и умножается на пользовательский коэффициент скорости - пауза замораживает всё целиком (и траекторию, и авто-вращение, завязанное на то же время).

Процедурный цвет вершины - линейное отображение локальной позиции в RGB: color = (v_local * 0.5 + 0.5) * tint. На 8 углах куба получаются 8 предельных цветов (чёрный, красный, …, белый), которые интерполируются растеризатором по граням.

Объекты Vulkan

VkBuffer + VmaAllocation - вершинный буфер (8 * vec3) и индексный (36 * uint16), оба в HOST_VISIBLE памяти через VMA, залиты через persistent-mapped указатель.

VkShaderModule - SPIR-V модули из cube.vert.spv / cube.frag.spv, собираемые из GLSL через glslc на этапе CMake.

VkPipelineLayout - объявляет один диапазон push constants (mat4 mvp + vec4 tint, 80 байт на стадию vertex).

VkPipeline (graphics) - собранный конвейер:

vertex input: один binding (stride = sizeof(vec3)), один атрибут (R32G32B32_SFLOAT);

input assembly: TRIANGLE_LIST;

rasterizer: CULL_MODE_BACK_BIT, FRONT_FACE_COUNTER_CLOCKWISE (с учётом Y-flip в проекции);

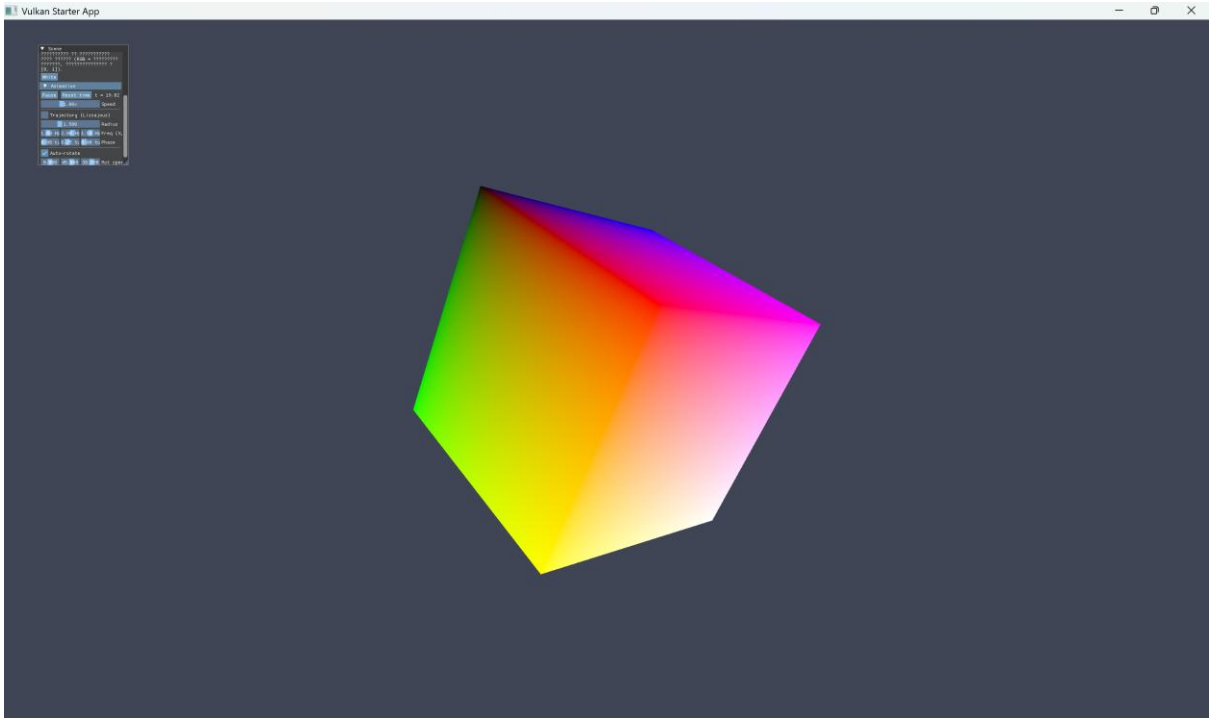
depth: TEST_ENABLE + WRITE_ENABLE, COMPARE_OP_LESS - на depth-attachment, созданный движком;

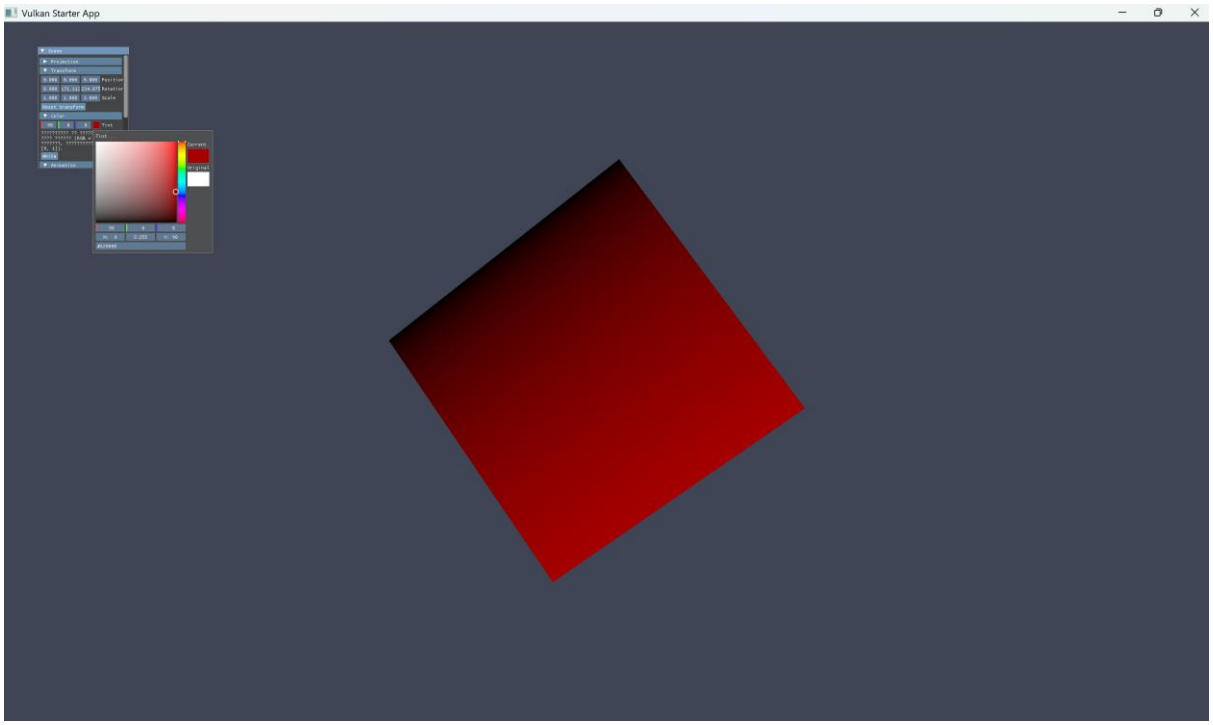
dynamic state: viewport + scissor (чтобы не пересобирать pipeline на ресайз окна).

VkCommandBuffer - внутри render() каждый кадр: begin -> vkCmdBeginRenderPass с двумя clear values (цвет + глубина) -> set viewport/scissor -> bind pipeline + bind VBO + bind IBO -> vkCmdPushConstants -> vkCmdDrawIndexed(36) -> end render pass -> end buffer.

Готовые для нас движком объекты - VkRenderPass, framebuffer со swapchain + depth-attachment, VMA allocator, semaphores/fence для синхронизации кадров - используются как есть.

Результаты





В окне рендерится цветной куб на тёмно-синем фоне. По умолчанию он движется по траектории Лиссажу (f = 1/2/1.5 Гц) и одновременно вращается вокруг Y и Z

Панель Scene слева управляет всем в реальном времени:

Projection - радио-переключатель Perspective/Orthographic переключает тип проекции налету: в перспективе куб уменьшается с расстоянием, в орто-проекции сохраняет размер; слайдеры FOV / Size /

Near / Far показывают, как меняется поле зрения, границы отсечения и перспективное искажение.

Color - ColorEdit3 с палитрой; при белом тинте видна исходная RGB-раскраска углов, при насыщенном красном весь куб становится градиентом от чёрного (угол (-1,-1,-1)) к красному (угол (1,1,1) при чисто-красном тинте). Виден эффект умножения на процедурный цвет

Выводы

Разобрался в следующих вещах.

Как данные движутся: vertex buffer -> vertex shader -> растеризатор -> fragment shader -> framebuffer.

Матрица MVP как композиция трёх независимых преобразований (model, view, projection) - каждое отвечает за свой слой (объект в мире / мир глазами камеры / камера на экране).

Clip-space и NDC: что именно должен выдать vertex shader в gl_Position, и чем Vulkan отличается от OpenGL (диапазон глубины [0, 1] вместо [-1, 1], инвертированная Y).

Зачем нужен depth-буфер и CULL_MODE_BACK_BIT - без них передние грани могут быть перекрыты задними, а невидимые треугольники зря нагружают fragment shader